

ARM Arithmetic and Logical Instructions

Arithmetic operations, bitwise logic, barrel shifter scaling, and immediate constant generation in AArch64

Administrative Check-In & Reminders

Progress Tracking

- Review prior problem sets on two's complement and performance metrics.
- Ensure all lab environments (QEMU / Compiler Explorer) are running smoothly for AArch64 testing.

Classroom Expectations

- Maintain the golden rule of classroom engagement.
- Focus on the underlying architectural logic rather than memorizing raw codes.

Topic 5 Technical Roadmap

Topic Objective

Examine how the AArch64 Instruction Set Architecture specifies arithmetic computation, bitwise logic manipulation, flexible barrel-shifter operations, and immediate constant handling.

- **Section 1:** AArch64 Arithmetic Instructions and Operand Sizing
- **Section 2:** Logical, Bitwise, and Shift Operations
- **Section 3:** Immediate Values, Constants, and Advanced Synthesis
- **Section 4:** Summary and Worked Self-Test Exercises

Foundations of Data Processing

The Role of Data-Processing Instructions

Data-processing instructions form the computational core of the CPU, transforming data held inside general-purpose registers without directly touching main memory.

- Operates entirely within the CPU core datapath.
- Separates compute instructions cleanly from memory load/store operations.
- Designed for uniform execution speed and clean pipelining.

AArch64 Arithmetic Instructions and Operand Sizing

Arithmetic Instructions in AArch64

AArch64 provides clean, uniform three-operand assembly syntax for computation:

- **Syntax Format:** `mnemonic destination, source1, source2`
- Operands can target 32-bit registers (`w0–w30`) or 64-bit registers (`x0–x30`).
- Arithmetic instructions execute directly within the CPU datapath using general-purpose registers.

Core Arithmetic Opcodes

- `add` / `adds`: Addition (with optional flag setting).
- `sub` / `subs`: Subtraction (with optional flag setting).
- `neg`: Negation ($0 - \text{operand}$).

Operand Sizing: W versus X Registers

The register prefix determines whether an operation treats data as 32-bit or 64-bit quantities:

32-Bit Operations (W registers)

- Uses registers like w0 through w30.
- Computes 32-bit results.
- **Zero-Extension:** Automatically zeroes out the upper 32 bits of the parent 64-bit X register.

64-Bit Operations (X registers)

- Uses registers like x0 through x30.
- Computes full 64-bit doubleword results.
- Native size for memory addresses and 64-bit variables.

Executing Basic Addition and Subtraction

Let's examine concrete examples of arithmetic manipulation in AArch64 assembly:

- `add x0, x1, x2` $\rightarrow x0 \leftarrow x1 + x2$ (64-bit addition)
- `sub x3, x4, x5` $\rightarrow x3 \leftarrow x4 - x5$ (64-bit subtraction)
- `add w0, w1, w2` $\rightarrow w0 \leftarrow w1 + w2$ (32-bit addition, upper 32 bits cleared)

Key Design Principle

Registers act as the primary operands for all arithmetic transformations; memory cannot be modified directly by an `add` or `sub` instruction.

The Hardwired Zero Register

AArch64 provides a special-purpose register that always reads as zero and discards writes:

- **xzr**: 64-bit zero register.
- **wzr**: 32-bit zero register.

Common Idioms Using Zero

- **Register Copy**: `add x0, x1, xzr` computes $x0 \leftarrow x1 + 0$, effectively copying `x1` into `x0`.
- **Register Negation**: `sub x0, xzr, x1` computes $x0 \leftarrow 0 - x1$, negating the value.
- **Comparison Alias**: Comparing a register against zero can be achieved using `subs xzr, x0, #0`.

Section 02

Logical, Bitwise, and Shift Operations

Logical and Bitwise Instructions

Bitwise instructions perform parallel boolean logic across all bits of a register independently:

- **and**: Bitwise AND (used for bit masking and clearing bits).
- **orr**: Bitwise OR (used for setting bits).
- **eor**: Bitwise Exclusive-OR (used for toggling bits or zeroing registers via `eor x0, x0`).
- **bic**: Bitwise Bit Clear (AND NOT: clears bits in destination specified by mask).
- **mvn**: Bitwise NOT (moves negated source into destination).

Practical Uses of Bitwise Logic

Bitwise instructions are fundamental for low-level system programming and hardware control:

Bit Masking (`and`)

Extracts specific bit fields by clearing unwanted bits.

- `and x0, x1, 0x0F` isolates the lower nibble of `x1`.

Bit Setting (`orr`)

Forces specific bits to 1 without altering others.

- `orr x0, x1, 0x80` sets bit 7.

Advanced Bitwise Opcodes: `bic` and `mvn`

Bitwise Clear (`bic`)

The `bic` instruction performs an AND operation of the first source with the bitwise NOT of the second source:

$$\text{destination} \leftarrow \text{source1 AND (NOT source2)}$$

- Ideal for clearing individual flag bits or clearing specific status masks.

Bitwise NOT (`mvn`)

The `mvn` (Move Not) instruction inverts all bits of the source operand before placing them in the destination:

$$\text{destination} \leftarrow \text{NOT source}$$

Register Zeroing Idioms

Clearing registers is a frequent operation in compiled code:

- While you can write `mov x0, #0`, compilers frequently emit exclusive-or idioms:

```
eor x0, x0, x0
```

- **Why use eor?**
 - XORing any value with itself always evaluates strictly to zero ($X \oplus X = 0$).
 - It avoids dependency on constant generation pools, executing cleanly and efficiently in hardware datapath pipelines.

Shift Instructions in AArch64

AArch64 provides dedicated shift instructions to move bits left or right:

- **lsl** (Logical Shift Left): Shifts bits left, filling low-order bits with zeros (multiplies by 2^n).
- **lsr** (Logical Shift Right): Shifts bits right, filling high-order bits with zeros (unsigned division by 2^n).
- **asr** (Arithmetic Shift Right): Shifts bits right while preserving the sign bit (signed division by 2^n).
- **ror** (Rotate Right): Shifts bits right and wraps shifted-out bits back into the high-order positions.

The AArch64 Barrel Shifter

Hardware Integration

AArch64 features a built-in hardware **barrel shifter** attached directly to the second operand input of data-processing instructions, allowing a shift to occur **in the exact same clock cycle** as the arithmetic operation.

- **Example Syntax:** `add x0, x1, x2, lsl #3`
- **Semantic Evaluation:** Computes $x0 \leftarrow x1 + (x2 \times 8)$ entirely in one single instruction cycle.
- Eliminates the need for separate standalone shift instructions in common array-indexing and scaling scenarios.

Barrel Shifter Scale Variations

The barrel shifter supports multiple modifier types on the final register operand:

- `lsl #imm` (Logical Shift Left by immediate)
- `lsr #imm` (Logical Shift Right by immediate)
- `asr #imm` (Arithmetic Shift Right by immediate)

Common Compiling Pattern

When C code multiplies a variable by a constant power of two (e.g., `val * 16`), compilers do not emit a slow `mul` instruction; they compile it directly into a shifted add or move:

```
lsl x0, x1, #4
```

Shift Amount Constraints

Shift operations in AArch64 have strict architectural boundaries:

- For 32-bit (W) register operations, shift amounts must range from 0 to 31.
- For 64-bit (X) register operations, shift amounts must range from 0 to 63.
- Attempting to shift by an amount equal to or greater than the register width results in undefined behavior or assembler errors.

Important Distinction

Logical shifts always pad with zeros, whereas arithmetic right shift (asr) replicates the MSB to maintain two's complement sign correctness.

Section 03

Immediate Values, Constants, and Advanced Synthesis

Handling Constants (Immediates)

Arithmetic and logical instructions frequently need to combine registers with constant values:

- **Immediate Operand:** A constant numeric value embedded directly inside the instruction word.
- In fixed-width 32-bit architectures like AArch64, instructions cannot contain arbitrary 64-bit constants because the instruction word itself is only 32 bits total.
- Special hardware mechanisms handle constant generation efficiently without requiring full 64-bit loads from memory.

Immediate Limits in Arithmetic Instructions

Arithmetic Immediates (add, sub)

For standard add and subtract instructions, immediate values are restricted to:

- Unsigned 12-bit integers ranging from 0 to 4095 (0x0xFFF).
- Optional hardware bit-shift modifier of 12 bits (multiplying the immediate by 4096), allowing efficient addition of larger aligned constants.
- Example: `add x0, x1, #1024` is fully valid.
- Adding an arbitrary 64-bit constant requires loading it into a temporary register first.

Logical Immediates and Bitmask Encoding

Logical instructions (`and`, `orr`, `eor`) handle constants using a sophisticated bitmask system:

- Instead of simple numeric ranges, logical immediates are formed by replicated bit patterns across the 32-bit or 64-bit word.
- Encoded using a combination of element size, rotation, and pattern repetition.
- Ensures that common bitmasks (such as byte-aligned flags or power-of-two selectors) can be embedded directly into single-cycle logical instructions.

Loading Large Constants: `movz` and `movk`

When a constant exceeds immediate limits, AArch64 uses explicit move instructions:

`movz` (Move Wide with Zero)

- Moves a 16-bit immediate into a specific 16-bit chunk of the register.
- Automatically zeroes out all other bits in the register.

`movk` (Move Wide with Keep)

- Moves a 16-bit immediate into a specific chunk.
- **Keeps** all other bits in the register untouched, enabling multi-step constant assembly.

Synthesizing Complex Expressions

Compilers combine arithmetic, logical instructions, and barrel shifts to evaluate complex expressions:

C Statement: `x = (a + (b « 2)) & 0xFF;`

Translated into efficient AArch64 assembly:

- ① Scale *b* and add to *a*: `add x0, x1, x2, lsl #2`
 - ② Mask lower byte: `and x0, x0, #0xFF`
- Notice how the barrel shifter merges the shift and add into a single hardware operation, reducing cycle count and instruction footprint.

Connecting Instructions to Condition Flags

Arithmetic and logical operations can update or test condition flags for control flow:

- **Arithmetic Flag Updates:** Using `adds` or `subs` updates the NZCV flags in PSTATE based on the result.
- **Logical Flag Updates:** Logical instructions like `ands` can update flags (specifically setting $Z = 1$ if the bitwise result is zero, and N based on the MSB).
- Crucial for checking bit masks or testing whether a register has become zero without needing a separate comparison instruction.

Performance Implications in Datapaths

Understanding instruction complexity directly impacts CPI (Cycles Per Instruction):

- **Single-Cycle Instructions:** Simple arithmetic (`add`), bitwise logic (`and`), and barrel-shifted operations execute in a single pipeline cycle on modern superscalar cores.
- **Multi-Step Sequences:** Loading massive 64-bit constants requires multiple instructions (`movz` followed by `movk`), increasing instruction count.
- Clean ISA design ensures that the most frequent operations map to single-cycle hardware pathways.

Summary & Self-Test

Topic 5 Comprehensive Summary

- **Arithmetic Instructions:** Execute addition, subtraction, and negation across 32-bit (W) and 64-bit (X) registers using uniform syntax.
- **Logical Instructions:** Execute parallel bitwise boolean operations (and, orr, eor, bic, mvn) for masking and setting bits.
- **Shift Operations:** Support scaling via lsl, lsr, asr, and ror, with hardware barrel-shifter integration for single-cycle scaling.
- **Constant Generation:** Managed through 12-bit arithmetic immediates, logical bitmasks, and multi-step wide moves (movz/movk).

Self-Test: Questions 1 & 2

Question 1: Operand Sizing

What is the structural difference between operating on 32-bit W registers versus 64-bit X registers in AArch64, and what happens to the upper bits of a register when a 32-bit instruction executes?

Question 2: Bitwise Logic

How would you use AArch64 logical instructions to clear bit 4 of register x0 while leaving all other bits completely unaffected?

Self-Test: Solutions 1 & 2

Solution 1

W registers access the lower 32 bits of the 64-bit X registers. When a 32-bit instruction executes, AArch64 automatically zero-extends the result by clearing the upper 32 bits of the parent register to zero.

Solution 2

You can clear bit 4 using a bit clear instruction with a mask containing a 1 at bit 4: `bic x0, x0, #(1 << 4)` (or using an explicit immediate mask).

Self-Test: Questions 3 & 4

Question 3: The Barrel Shifter

What is the architectural advantage of the AArch64 built-in barrel shifter during data-processing instructions?

Question 4: Constant Loading

Why can't an AArch64 arithmetic instruction contain an arbitrary 64-bit constant directly, and how do compilers solve this limitation?

Self-Test: Solutions 3 & 4

Solution 3

The barrel shifter allows a source operand to be shifted or scaled immediately as part of the arithmetic or logical instruction in a single clock cycle, avoiding extra standalone shift instructions.

Solution 4

Because instruction words are fixed at 32 bits, there is not enough room to store both an opcode, register identifiers, and a full 64-bit constant. Compilers solve this using 12-bit immediates, logical bitmasks, or wide move sequences (`movz` / `movk`).

Review: Instruction Summary Table

Instruction Category	Common Opcodes	Primary Architectural Purpose
Arithmetic	add, sub, neg	Integer computation across W/X registers
Logical / Bitwise	and, orr, eor, bic	Masking, setting, clearing, and toggling bits
Shifts	lsl, lsr, asr, ror	Multiplying/dividing by powers of two
Constants	movz, movk	Constructing wide constants across register chunks

Review Exercises & Practice Problems

Self-Directed Practice

To solidify your understanding of Topic 5 concepts, try the following exercises in your lab environment:

- Write an AArch64 assembly snippet to extract bits 12 through 15 from register `x1` into `x0`.
- Use a barrel-shifted add instruction to compute $x5 = x6 + (x7 \times 32)$ in a single cycle.
- Explain why `eor x1, x1, x1` is preferred by compilers over `mov x1, #0`.

Wrap-Up and Next Steps

Moving Forward to Topic 6

Now that we understand arithmetic, logic, shifts, and constant generation, our next session will explore:

- **Memory Access Instructions:** Loading and storing data using `ldr` and `str`.
- **Advanced Addressing Modes:** Post-index, pre-index, and register-offset memory mapping in AArch64.

End of Topic 5 — Questions & Discussion